BATCH 9 · WHAT WE DID, WHY WE DID IT, HOW WE DID IT, AND WHAT IT IS WORTH

The Whole Story, in Plain English

Credit Pre-Screening Engine — Loan Intake Optimisation · MUC02 · Path 2, KLU Batch

Read this if you want the whole project in one sitting. No jargon. Every technical word is explained the first time it appears. The numbers in bold are the ones that matter — they are the same numbers in the notebook, the slides and the summary.

1. The story — a bank with a queue problem

A mid-size private bank in India gives out **unsecured personal loans**. Unsecured means there is no gold, no house, no car behind the loan — if the customer stops paying, the bank has nothing to take back. So the bank has to be careful about who it says yes to.

Its customers are all sorts of people: salaried employees, self-employed professionals like doctors and architects, small business owners, and gig or contract workers. They live in big metros and in smaller Tier 2 and Tier 3 towns.

Every month, about **3,000 applications** arrive. And here is the thing that started this whole project: **every single one of them is read by a human being.** A credit analyst opens the file, checks the income, checks the credit score, checks the existing loans, makes a decision and writes it up. That takes about **25 minutes**.

3,000 applications × 25 minutes = **1,250 analyst hours every month.**
That is roughly **eight people doing nothing but reading application forms**, all month, every month.

Why that is a problem

The problem is not really the number of applications. The problem is **what is sitting in the queue.**

- About **47% of applications get declined.**
- A large share of those are obvious within the **first minute** — the credit score is far below anything the bank accepts, or the person's existing EMIs already swallow most of their salary.
- But the analyst still spends the full 25 minutes on it. Open, check, write up, close.

That creates three different problems for three different groups of people, and this is worth saying out loud because a good analysis has to serve all three:

Who	What goes wrong for them
The bank	It cannot take in more applications without hiring more analysts. Growth costs headcount.
The analysts	Their working day is spent on obvious 'no's, instead of the genuinely difficult borderline files where their judgement actually adds something.

Good customers	They wait in a queue behind applications nobody needed to read. Some of them give up and go to a competitor.

So the Head of Retail Lending asked the Credit Policy Analyst one direct question:

> "**Can we use our own history to build a pre-screening model, so analysts spend their time on applications that genuinely need judgement?**"

That is the question we were given. Note what it is **not**. It is not "build the most accurate model you can." It is "build something useful that a Credit Policy Analyst can defend to her boss, to the analysts whose jobs change, and to the customers it affects."

2. Why we framed it before we coded it

Before opening the data file, the four of us spent 20 minutes filling in a **CBIM Problem Canvas** — six questions you answer before you write a single line of code. It sounds like paperwork. It is not. It changed two things about how we worked.

The first change: we knew what success meant

Our instinct was that success meant "high accuracy." The canvas forced a better answer. Success is: **no more than 5% of the applications we auto-decline may be people the credit team would have approved** — and the volume we free up has to be stated in **applications and analyst hours per month**, not in percentages.

That is a very different target, and it changed the whole project. A model can be 95% accurate and still be unusable if the 5% it gets wrong are all good customers.

The second change: we understood what our data actually is

> **The `decision` column is what the credit team decided in the past. It is NOT whether the customer repaid the loan.**
>
> The bank never recorded repayment. So our model can only ever learn to **copy past analyst judgement**, including any inconsistency or bias in it. When we later say "4.7% wrong", we mean "4.7% disagreed with an analyst" — not "4.7% were bad credit risks."

This is the deepest limitation in the entire project, and we found it **before** we started, not after. That is the whole value of doing the canvas first.

3. How we worked — seven steps, four people

Step	What it does	Who owned it
5.1 Clean	Find and fix the problems in the raw file	Mohammed Mutee Ruknuddin
5.2 Find the drivers	What does the credit team actually do today?	Mohammed Saalif Udyawar
5.3 Build affordability	Create the FOIR feature that was not in the file	Mohammed Saalif Udyawar
5.4 Train the model	Logistic regression, and read what it learned	Mulla Javeed
5.5 Choose the threshold	Where do we draw the auto-decline line?	Mulla Javeed
5.6 Check fairness	Does the model treat men and women differently?	Nagothu Varshitha
5.7 Brief the bank	What we would actually say to the Credit Policy Analyst	Nagothu Varshitha

The order matters because each step feeds the next. The cleaning decides what the features look like, the features decide what the model can learn, the model decides where the threshold falls, and the briefing has to defend all four.

4. What we did, and why — step by step

Step 5.1 — Cleaning: three problems, three decisions

The file has **12,000 applications and 19 columns**. Three things in it needed a decision, and we wrote down the reason for each one, because a reviewer at the bank has to be able to read our notes and agree.

1. **441 incomes were missing.** We checked where they were missing from, and they were not random at all — about **13% of business owners** and **11% of self-employed professionals**, but under **1% of salaried** people. That makes sense: someone without a salary slip has no single clean number to declare. **The missingness is itself information.** We filled each one with the median income of that person's own employment type, not a single overall median — because gig workers earn ₹28,000 and self-employed professionals earn ₹75,000, so one global number would have been wrong for everybody.
2. **1,039 credit scores were blank.** This is the one most people get wrong. We checked the credit history column and it was **zero months for every single one of them** — and there was no row anywhere in the file with zero history but a score present. These are **new-to-credit** people: they have never borrowed, so the credit bureau has nothing to score. They are not errors. They are 8.7% of the bank's applicants, they are younger than average, and the bank approves them **53.9%** of the time. We kept them and flagged them with a new column, `is_ntc`.
3. **12 incomes were impossible** — over ₹6,00,000 a month, all salaried. Almost certainly someone typed an annual figure into a monthly box. How do we know? Because dividing every one of them by 12 puts **all twelve** back into a normal salary range. A data-entry explanation that works for all 12 cases beats 12 genuine six-lakh salaries. We divided by 12 — and we did it **first**, before calculating any medians, so the bad values could not pollute everything downstream.

The result: we kept all 12,000 rows. A single line of `dropna()` — which is what most people reach for — would have deleted about 9% of applicants: the young first-time borrowers and the self-employed. **Those are exactly the customers this bank is trying to grow.**

Step 5.2 — What does the credit team actually do today?

Before building anything, we wanted to know what the humans already do. If the model later finds the same patterns, we have evidence it learned real credit policy rather than noise.

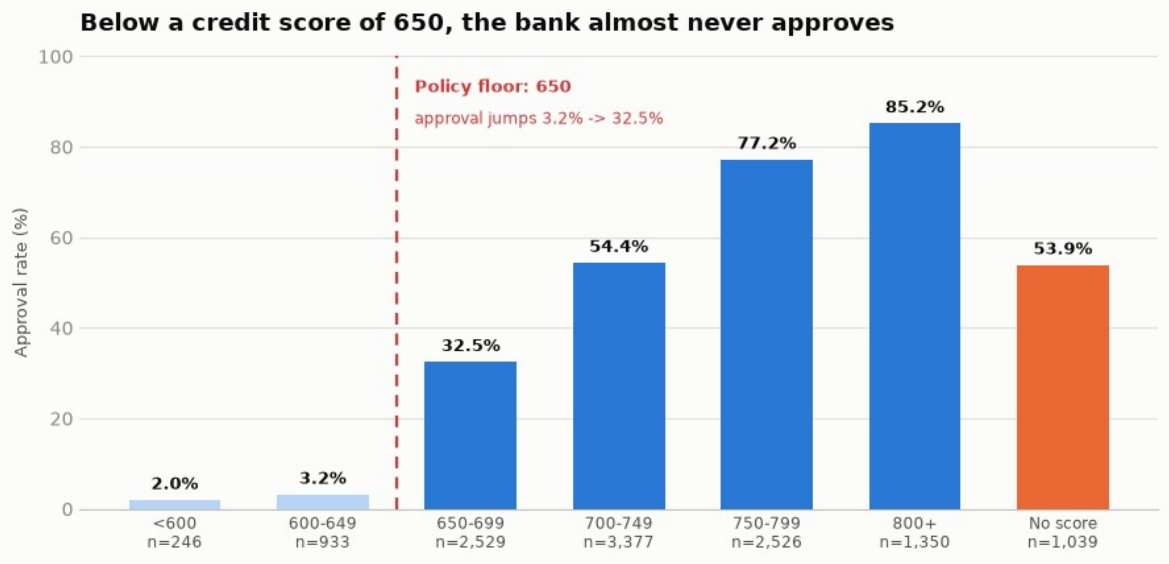


Figure 1 — Approval rate by credit score band. The jump at 650 is a rule, not a gradient.

Finding 1: the bank has a credit score rule that nobody wrote down. Below 650, approval is **3.0%** across **1,179** applications. Immediately above it, **32.5%**. At 800 and above, **85.2%**.

A tenfold jump across one single point on a scale is not a gradient — it is a rule. No one has written "650" into the credit policy document, but every analyst applies it. **That is the safest thing in the whole dataset to automate**, because the machine is not being asked to exercise judgement; it is being asked to repeat a decision the credit team has already made the same way 1,179 times.

Notice also the orange bar: people with **no credit score at all** sit right in the middle at **53.9%** — basically the house average. There, the credit team is clearly using **judgement**, not a rule. Remember that bar. It becomes the most important caveat in the whole project.

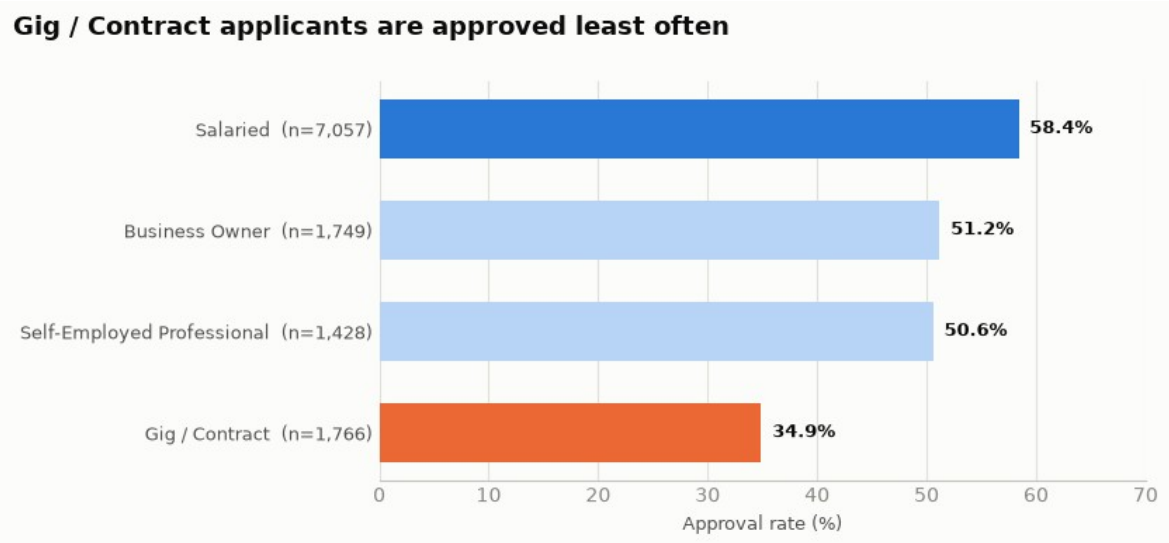


Figure 2 — Approval rate by employment type. A 23-point spread between salaried and gig workers.

Finding 2: gig and contract workers are approved least — **34.9%** against **58.4%** for salaried applicants. But the job label is not the cause. Gig applicants have a **median income of ₹28,000 against ₹50,000** for salaried applicants, plus shorter job tenure. The same loan therefore eats a much bigger share of their income. **Employment type is standing in for income stability**, not for the job title itself.

Step 5.3 — Building the number that was not in the file

Nothing in the raw data measures **affordability** — whether the person can actually carry the new EMI on top of what they already pay. Indian lenders measure this with **FOIR**, the Fixed Obligation to Income Ratio:

> **FOIR = (EMIs they already pay + the new EMI) ÷ monthly income**
>
> A FOIR of 0.60 means 60% of the person's salary would go to loan repayments.

We had to build this column ourselves. It turned out to be **the single strongest predictor in the entire dataset.**

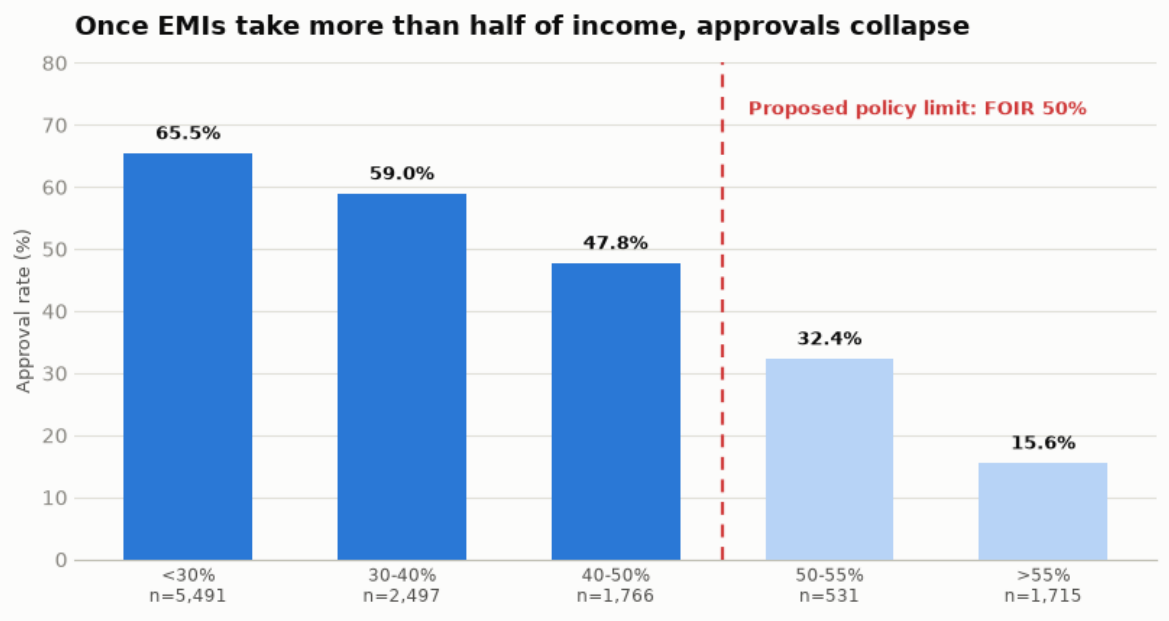


Figure 3 — Approval rate by FOIR band. Approvals collapse once EMIs pass half of income.

Finding 3: approvals fall off a cliff past 50% FOIR. The decline is gentle up to 50% — **65.4%, 59.0%, 47.4%**, about six points a band. Then it collapses: **47.4% → 31.7% → 15.6%**. Approval more than halves between one band and the next.

Written as credit policy, that becomes: **a FOIR limit of 50%**, with **50–55% as a referral band** that needs explicit justification, and **above 55% a decline** unless there is a documented reason. This is not a new rule we invented — it is the bank's own unwritten rule, made explicit so every analyst applies it the same way.

Step 5.4 — Building the model

We used **logistic regression**. In plain terms: it looks at all the features of an application at once and produces a **probability between 0 and 1** that the application should be declined. It is a simple model, and that is a feature, not a limitation — the credit team has to be able to read what it learned and agree with it. A model nobody can explain is a model nobody can defend to a regulator.

Three choices worth recording:

- **The target is Declined = 1.** Auto-declining is the action we want to take, so the class we care about should be the positive one.
- **Gender, city and the decision column were excluded.** Gender because a bank cannot defend a credit decision that uses it. The decision column because that is the answer — including it would be cheating.
- **12,000 applications were split 80/20** — 9,600 to learn from, **2,400 held back** and never shown to the model, so we can test it on applications it has never seen.

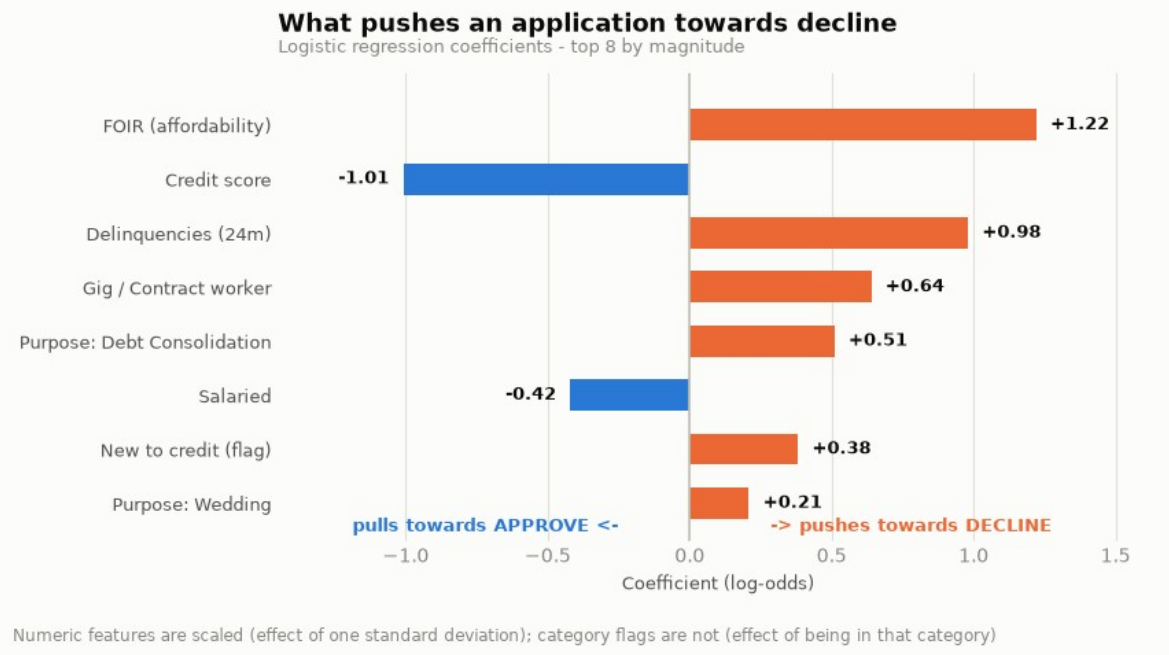


Figure 4 — What the model learned. Orange pushes towards decline, blue pulls towards approval.

The three strongest signals are FOIR (+1.22), credit score (−1.01) and missed payments in the last 24 months (+0.98).

And here is the result that matters most. Those are **exactly the three drivers Saalif found by hand in Steps 5.2 and 5.3, in the same order of importance.**

We never told the model about the 650 cliff or the 50% FOIR line. **It rediscovered both on its own.** That is the strongest evidence available that it learned the bank's real credit policy rather than a fluke of this particular six months — and it is what makes it defensible to a credit analyst who has been doing this for twenty years.

How you judge a model like this

The headline number is **79.1% accuracy**, and that number is close to useless on its own. Here is what actually matters.

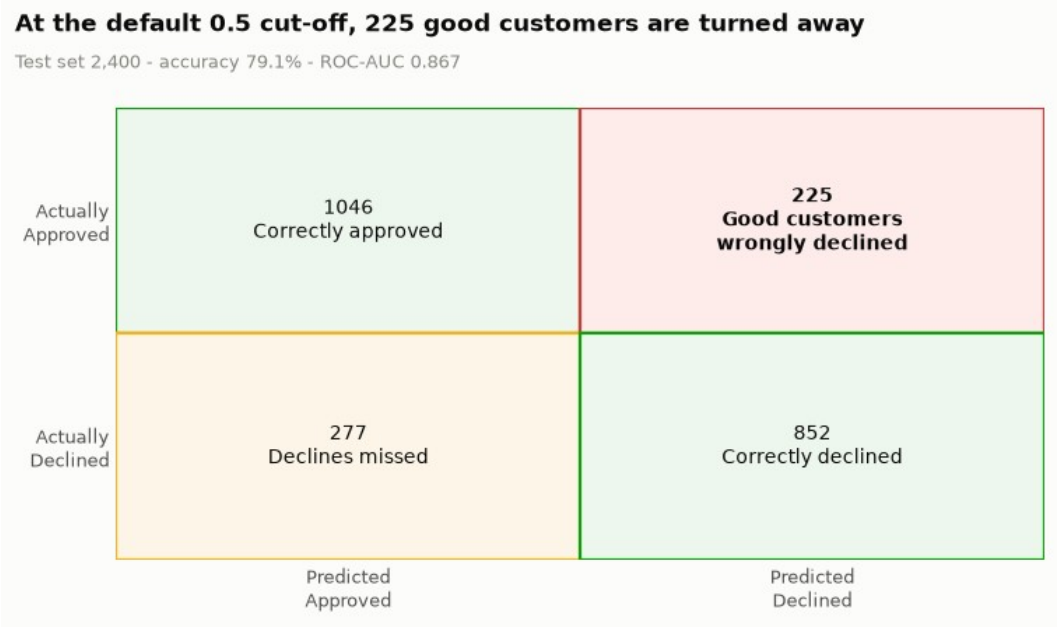


Figure 5 — The confusion matrix: what the model said against what really happened.

A **confusion matrix** is just a 2×2 table comparing the model's prediction with reality:

Box	Count	What it means in real life
True Negative	1,046	Model said approve, the team approved. Correct.
False Positive	**225**	Model said decline, but the team had **approved** them. **A real customer, turned away by a machine.** This is the expensive mistake.
False Negative	277	Model said approve, the team declined. Costs 25 minutes of an analyst's time. Cheap.
True Positive	852	Model said decline, the team declined. Correct.

Two more words you will hear:

- **Precision (79.1%)** — of everyone the model declines, how many really were declines. It answers *"when I acted, was I right?"*
- **Recall (75.5%)** — of all the real declines, how many the model caught. It answers *"did I catch everything?"*
- **ROC-AUC (0.867)** — a single score for how well the model separates the two groups across every possible cut-off. 0.5 is a coin toss, 1.0 is perfect.

The two errors are not equal, and that asymmetry is the entire use case. A false negative wastes 25 minutes. A false positive loses a customer, their lifetime value, and possibly a complaint the bank cannot defend.

"The model is 79% accurate" describes the model. "**225 applicants we would have approved are turned away**" describes the business. The bank only cares about the second one.

Step 5.5 — The decision at the heart of the whole project

A logistic regression does not output a decision. It outputs a **probability**. Turning that into an action needs a **cut-off**, and this is where most people stop thinking: Python's `predict()` silently uses **0.5**.

0.5 is a statistical convention. It has nothing to do with lending. At 0.5, this model auto-declines **44.9%** of applications — and **one in five of those (20.9%) is someone the credit team approved.** No lending unit on earth would sign that off.

So the bank states its own risk appetite instead, and the model's only job is to show where that line falls:

> **No more than 5% of auto-declined applications may be ones the credit team would have approved.**

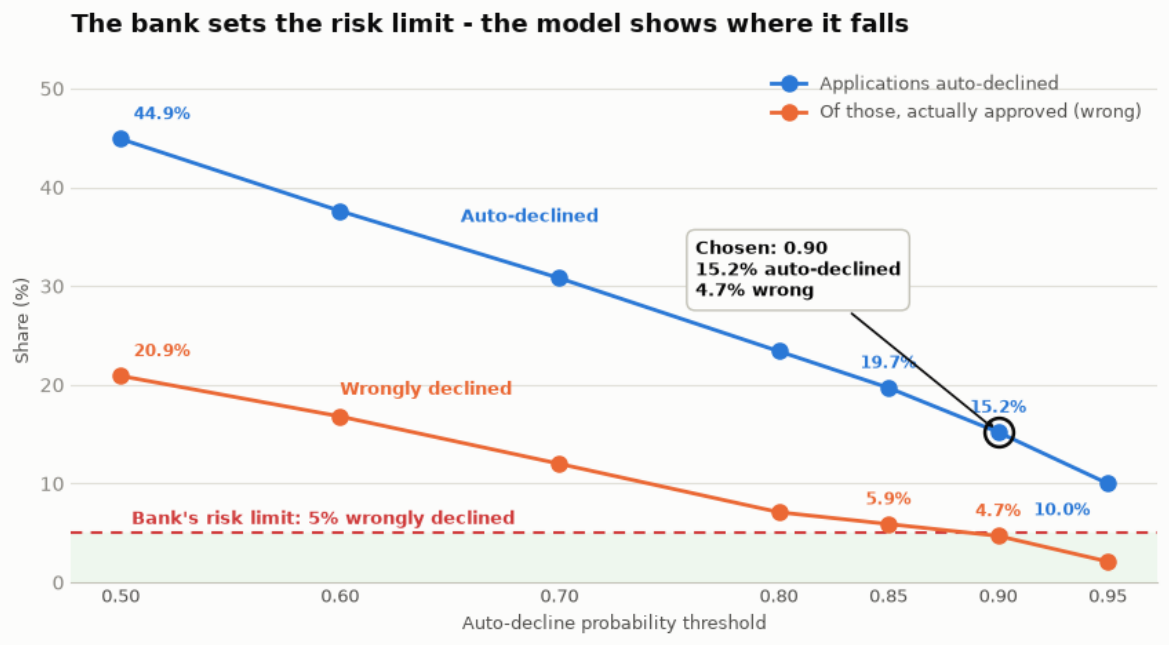


Figure 6 — The trade-off. The bank sets the red line; the model shows where it is crossed.

Threshold	% auto-declined	% wrongly declined	Inside the 5% rule?
0.50	44.9%	20.9%	No
0.70	30.8%	12.0%	No
0.80	23.4%	7.1%	No
0.85	19.7%	5.9%	No
0.90	**15.2%**	**4.7%**	**Yes — and it is the lowest that is**
0.95	10.0%	2.1%	Yes, but removes far less

We chose 0.90. And note that we chose the **lowest** threshold that passes, not the safest one. Every step above 0.90 buys safety the bank has already said it does not need, and pays for that

safety in lost volume. 0.95 is comfortably safe at 2.1% — and removes about 110 fewer applications a month for no benefit.

What 0.90 is worth every month
15.2% of applications auto-declined · **about 456 applications** · **about 190 analyst hours** (15% of the unit's 1,250-hour load, roughly one full-time analyst) · about **21 wrong declines a month** · **30.7%** of all declines caught.

One honest comparison. A single rule — `credit_score < 650` — would flag 9.8% of applications with only 3.0% wrong, with no model at all. The model adds about **5 percentage points of extra volume**, worth roughly 160 applications and 67 analyst hours a month, from cases where FOIR and missed payments justify a decline that the score alone would not. That is a real gain, and it is the answer to the obvious question from the Head of Retail Lending: *“why not just set a score floor?”*

The blind spot we went looking for

Overall accuracy is an average, and averages hide things. We split the test set in two and found something the headline number completely conceals:

> The model is **81.2% accurate on applicants who have a credit score**, and **56.7% on applicants who do not.** For that second group it is **barely better than tossing a coin.**

And the reason is **structural, not statistical** — which means no amount of better modelling fixes it. Think about what it means to have never borrowed:

- `delinquencies_24m` is **0** — you cannot miss a payment on a loan you never had.
- `num_active_loans` is **0** — for the same reason.
- `existing_emi` is **0** — again, for the same reason.
- `credit_score` is a **median value we filled in ourselves**, not real information.

So **three of the model's four strongest features carry no information whatsoever about this group**, and the fourth is invented. We confirmed it two more ways: a model trained only on new-to-credit applicants reaches an AUC of **0.632** against 0.867 for the main model, and at our 0.90 threshold, **12 of the 364 auto-declined applications were new-to-credit — and 6 of those 12 were people the bank had approved.**

So we proposed a second design: **never auto-decline an applicant with no credit history.** Route them to a human, always. And the result is better on both counts at once:

	Design A — one lane for everyone	**Design B — no credit history always sees a human**
Threshold	0.90	**0.88**
Applications auto-declined	15.2%	**16.4%**
Wrongly declined	4.7%	**4.1%**
Per month	~456 applications · ~190 hours	**~492 applications · ~205 hours**

More volume and less risk at the same time. Removing the one segment the model cannot read lets us lower the cut-off and still sit further inside the bank's limit. There is no trade-off to argue about.

It is also far easier to defend. *“We never auto-decline someone with no credit history”* is a sentence the Head of Retail Lending, a regulator and a declined customer can all follow — and it is operationally trivial: a null check on one column before the model is even called.

Step 5.6 — Is it fair?

We did not use gender as a feature. That on its own does not make a model fair, because a model can reach a protected attribute through anything correlated with it. So we measured four things instead of assuming.

4. **There is a real gap in the bank's history. 54.1%** of male applicants approved against **50.4%** of female applicants — a **3.7-point** gap.

5. **But it mostly disappears when you hold employment type constant.** Inside Gig/Contract the gap is **0.5 points**; inside Business Owner it is **−0.6**, actually favouring women. The aggregate gap comes from **which groups women are in**, not from different treatment within those groups. (Statisticians call this Simpson's paradox.)
6. **Two compositional differences explain it.** Female applicants have a **median income of ₹45,000 against ₹55,000** — about 18% lower, which means a higher FOIR on the same loan — and they are **almost twice as likely to be gig or contract workers (21% vs 12%)**, the lowest-approval category. Credit scores (723 vs 727) and new-to-credit rates (8.5% vs 8.7%) are essentially identical, which rules those out.
7. **Our model does not widen the gap.** At the 0.90 threshold it auto-declines **15.6% of women and 15.0% of men** — a 0.6-point gap against 3.7 points in the human decisions. Inside the auto-declined group, the wrong-decline rate is **3.4% for women against 5.2% for men**, so the errors do not fall more heavily on female applicants.

Our position. Gender should **never** be a feature in a credit model — it is indefensible to a regulator and to a customer, whatever it does for accuracy. But **leaving it out is necessary, not sufficient**: income and employment type are strong proxies, and a model built only on those can reproduce a gender gap without ever seeing the column.

And one more honest point: **the model inherits whatever bias is in the target.** It copies historical analyst decisions, so parity with the analysts is parity with a possibly-biased baseline. “No worse than the humans” is not the same as “fair.”

5. What it is worth — the impact

Five ways this turns into money. Three we measured; two the bank can now go and measure.

	Lever	The number	Status
1	**Capacity without hiring**	190 hours a month = **2,280 hours a year ≈ 1.2 full-time analysts.** The unit can absorb about **15% more applications** with the team it already has.	Measured
2	**Time moves to where money is made**	Those hours shift from obvious 'no's onto **borderline files** — the ones where an approval is actually won or lost. A clear decline earns the bank nothing.	Measured
3	**Faster answers for good customers**	**456 files leave the queue every month**, so everyone behind them moves up. Fewer applicants give up and go to a competitor while waiting.	Volume measured; drop-off needs data
4	**One consistent written credit policy**	A **50% FOIR limit** applied the same way by every analyst, every day — less inconsistency, fewer over-leveraged loans written by accident.	Policy gain
5	**Protecting tomorrow's customers**	The new-to-credit carve-out saves about **6 young first-time borrowers a month** from a wrong machine rejection. These are the longest-lifetime customers a bank has.	Risk avoided

Putting a rupee figure on it — with the assumption stated out loud.
2,280 analyst hours a year is our number, from our analysis. At an **assumed** fully-loaded analyst cost of **₹500 an hour**, that is roughly **₹11.4 lakh a year of capacity released** — before counting a single rupee from faster decisions or recovered gig-worker applications.

The ₹500 an hour is our assumption, not the bank's figure. The hours are ours; the rate has to come from Finance. We would rather state the assumption than hide it inside a number.

6. What we recommend, in order

8. **Shadow-run it for one month.** Score live applications and record what the model *would* have done — without acting on it. Compare against the analysts' real decisions. *(Month 1)*
9. **Launch at 0.90, with the carve-out.** Auto-decline at 0.90, and never auto-decline an applicant with no credit history. Move to 0.88 once the shadow month confirms the rates. *(Month 2)*
10. **Write the FOIR limit into policy** — 50% limit, 50–55% referral band, above 55% a decline unless justified. *(Month 2)*
11. **Start capturing repayment outcomes.** Today we only know who was approved, not who repaid. This is the single biggest upgrade available to the next version. *(From day 1)*
12. **Stand up monthly monitoring** — auto-decline rate and wrong-decline rate, overall and by gender, with a written trigger to review the threshold. *(Ongoing)*
13. **Recalibrate every quarter.** The threshold comes from 2,400 test applications and the credit policy itself will drift. *(Quarterly)*

The ask is small and the exit is easy. A shadow month costs nothing and can be stopped at any point. If the numbers hold, the bank gets 190 hours a month back. If they do not, nobody was ever auto-declined.

7. What we would want before this goes live

The target is a human decision, not repayment

The model learns to copy past analyst judgement. "4.7% wrong" means "4.7% disagreed with an analyst" — **not** "4.7% were bad credit risks." This is the deepest limitation in the whole analysis, and the reason repayment data is our top recommendation.

The threshold is estimated from 2,400 applications

4.7% carries sampling error. At ~456 auto-declines a month, expect real month-to-month movement — which is why we recommend recalibrating quarterly rather than setting the number once.

Our own cleaning choices move the answer

We tested nine defensible combinations of the Step 5.1 decisions. Accuracy barely moves (78.6% to 79.1%) — but the **recommended threshold moves between 0.85 and 0.90**, worth about 160 applications and 65 analyst hours a month. That is why the cleaning log exists: it is the audit trail for a business number.

A model is only as current as the policy it learned

If the credit team changes how it lends, the model keeps applying the old policy until somebody retrains it.

Where we land, in one sentence: auto-decline at 0.90, never auto-decline someone with no credit history, and shadow-run it for a month first — which frees about **190 analyst hours a month** at a wrong-decline rate of **4.7%**, inside the limit the bank set itself.